

Frida Ndingwan

DATA ENGINEER ·
PLATFORM ENGINEER

CONTACT

📍 Dallas, TX (Remote)
✉️ ndingfrida@gmail.com
☎️ (470) 734-7039
in [linkedin.com/in/frida-ndingwan](https://www.linkedin.com/in/frida-ndingwan)

TECH STACK

Languages

Python, PySpark, SQL

Data Platforms

Databricks, Delta Lake, Apache Iceberg, MLflow, Unity Catalog

Databases

PostgreSQL, Oracle, Amazon RDS, Aurora

Orchestration

Apache Airflow, Azure Data Factory, Databricks Jobs, GitHub Actions

Storage & Processing

Apache Spark, AWS S3, Parquet, ETL/ELT

Cloud

AWS, Azure, GCP

AWS Services

IAM, Secrets Manager, Parameter Store, S3, RDS, Aurora, DynamoDB, EC2, Lambda, Fargate, ECS, ECR, EKS, CloudWatch

Governance & DevOps

CI/CD, GitHub Actions, Jenkins, Docker, Kubernetes (eksctl, kubectrl, Helm), Prometheus, Terraform, RBAC

EDUCATION

M.S. in Financial Data Analytics

Catholic University (CUIB) · 2019

Professional Profile

Data and platform engineer operating an enterprise Databricks Lakehouse across **8 workspaces, 90 catalogs, and 100+ service principals**, supporting **12 data teams (~90 users)**. Experienced in engineering production-grade ingestion pipelines, administering Unity Catalog governance at scale, and building platform observability. Backed by a strong DevOps and cloud-infrastructure background — building CI/CD pipelines, Terraform Infrastructure-as-Code, and containerized workloads — with deep expertise in identity & access management, data quality, and automation across multi-cloud environments.

Professional Experience

Data / Platform Engineer

Oct 2024 – Present

Mountain Digital Systems · Texas, TX (Hybrid)

PLATFORM OPERATIONS & ADMINISTRATION

- Operated an enterprise **Databricks Lakehouse** spanning **8 workspaces, 90 catalogs, and 100+ service principals**, supporting platform administration across Unity Catalog governance, identity & access, compute, ingestion pipelines, and observability for **12 data teams**.
- Resolved **70–80 platform tickets/month** covering user/group provisioning, Unity Catalog access grants, workspace/compute requests, and incident response across dev, QA, and prod workspaces.
- Administered Unity Catalog governance: schema creation, ownership/RBAC grants across commercial_lakehouse and delivery catalogs, external-metadata enablement, and service-principal access for production applications.
- Managed identity & access at scale — onboarded **~90 users across 12 teams** into correct workspaces and groups with least-privilege grants.

DATA INGESTION PIPELINE ENGINEERING

- Supported the production ingestion framework delivering **PostgreSQL/Oracle → Apache Iceberg** extraction pipelines, ingesting **~30 tables per week** into the Lakehouse.
- Diagnosed and resolved recurring OutOfMemory pipeline failures — cut failures from **~8 per week to near zero** via memory-aware concurrency and right-sized container compute, reducing execution time by 15%.
- Hardened data correctness: resolved mixed-case/special-character Postgres column issues, added Parquet output handling and a validation step, and set PYTHONUNBUFFERED across job definitions for reliable logging.
- Built scheduled production pipelines (hourly + daily) and migrated datasets to dedicated **S3 buckets** with automated bucket provisioning in CI/CD.
- Designed, developed, and maintained additional Apache Spark pipelines within Databricks for large-scale structured and unstructured data ingestion; built and optimized ETL/ELT workflows and monitored cluster configuration, scaling, and cost.

OBSERVABILITY & ARCHITECTURE

- Supported a standardized Databricks Monitoring Framework covering **72 production pipelines** — tracking job status/duration/failures, cluster utilization & cost, data-quality validation, and alerting integration.
- Monitored the observability data layer — job_health_metrics, dq_results, cluster_usage, data_freshness (SLA), and alerts_sent tables with 90-day retention and analyst-read / service-principal-write access.
- Defined a data pipeline integration architecture to productionize customer onboarding logic through **GitHub Actions** workflows; maintained data quality via validation checks, testing frameworks, and failure/bottleneck monitoring.

AWS CLOUD INFRASTRUCTURE

- Built and deployed containerized **AWS Lambda** extraction services (Docker image-based) to ingest source data into the Lakehouse, with parameterized invoke/describe/deploy workflows wired into CI/CD.
- Managed credentials in **AWS Secrets Manager** — provisioned per-source secrets, standardized shorter secret paths, and granted scoped service-principal retrieval for production jobs.
- Engineered ingestion from **RDS PostgreSQL** and **Aurora** clusters into Apache Iceberg, handling connection management, schema/column edge cases, and re-ingestion of large multi-table datasets.
- Provisioned and administered **S3 buckets** via Infrastructure-as-Code with **Terraform** — automated existence checks and creation of only-missing buckets in CI/CD, with dedicated hyphenated bucket naming for prod.
- Applied **IAM** roles and least-privilege policies across service principals, Lambda execution roles, and cross-account access; right-sized EC2-backed container compute (scaling to 2xlarge) to eliminate ~8 weekly OOM failures and cut execution time 15%.

ML & ADVANCED ENGINEERING

- Developed and maintained **Delta Lake** tables for ACID-compliant storage; productionized ML models with **Databricks MLflow** in collaboration with data science teams.
- Implemented **Unity Catalog** for governance, access control, and metadata management; wrote and optimized complex **PySpark and SQL** queries for large-scale analytics.
- Integrated Databricks workflows across **AWS, Azure, and GCP**; automated orchestration via Databricks Jobs, Apache Airflow, and Azure Data Factory, and implemented CI/CD for notebooks and workflows.

AWS DevOps Engineer

Nov 2019 – Sep 2024

Anmute Technologies · Remote

- Provisioned and managed **AWS** infrastructure as code with **Terraform** (HCL) — VPCs, **EC2**, **S3**, **IAM**, and **RDS** across **3 environments** (dev/staging/prod), using reusable **modules** and applying safe configuration changes (e.g., instance-type upgrades) via terraform plan/apply.
- Configured **remote Terraform state** in an encrypted **S3** bucket with **DynamoDB** state locking, enabling safe, collaborative infrastructure changes without state conflicts.
- Automated build, test, and deployment through **CI/CD pipelines** in **GitHub Actions** and **Jenkins** (terraform init → plan → apply), cutting manual release effort and reducing deployment time from hours to minutes.
- Containerized applications with **Docker** — authored Dockerfiles and multi-container **Docker Compose** stacks, published images to **Amazon ECR**, and deployed on **Amazon ECS** following least-privilege (non-root) practices.
- Provisioned **Amazon EKS** clusters with **eksctl** (3 managed worker nodes) and deployed workloads via **kubectrl** — nginx pods exposed through a **LoadBalancer** service — and validated **high availability** by terminating nodes and confirming EKS self-healed pods with zero downtime.
- Packaged Kubernetes deployments as reusable **Helm charts** (templates + values.yaml) and deployed **Prometheus** via Helm for out-of-the-box cluster metrics and alerting.
- Developed event-driven **serverless** functions on **AWS Lambda** (with **Fargate** for containerized tasks), triggered by AWS service events, to remove server management overhead and pay only for usage.
- Centralized credentials and configuration in **AWS Secrets Manager** and **Systems Manager Parameter Store**, eliminating hard-coded secrets and enforcing scoped retrieval across environments.
- Set up monitoring, logging, and alerting with **CloudWatch** to improve visibility into application health and speed up incident response.
- Applied least-privilege **IAM** policies and right-sized compute resources, tightening access control and reducing monthly AWS spend.